

AI 개발 업무 자동화 시스템

기획안 (Draft v0.1)

프로젝트명(가칭): WorkPilot AI — 픽셀 오피스 UI

업무 요청 → 분석 → 작업 분해(WBS) → 담당자 배정 → 일정 생성 → 진행 추적 → 지연 대응 → 회의 요약

2026-08-11

1. 개요

1.1 배경 및 목적

PM이 업무를 지시하면 팀원들이 요구사항을 해석하고, 작업을 쪼개고, 담당자를 정하고, 일정을 짜고, 진행 상황을 챙기는 일련의 과정은 여전히 대부분 사람이 수작업으로 처리한다. 이 과정에서 요구사항 해석의 편차, 작업 분해 누락, 배정 불균형, 일정 지연 인지 지연 등의 문제가 반복적으로 발생한다.

WorkPilot AI는 "회원이입 시스템을 만들고 테스트까지 진행해줘"와 같은 자연어 업무 지시를 받아 AI가 요구사항을 분석하고, 실행 가능한 작업 단위(Task)로 자동 분해하고, 팀원 역량과 부하를 고려해 담당자를 추천·배정하고, WBS와 일정을 생성하고, 이후 진행 상황을 지속적으로 추적하며 지연을 감지해 담당자 알림과 일정 재조정안을 제시하는 개발 업무 자동화 시스템이다. 회의 음성/텍스트를 입력하면 이를 요약해 업무 맥락에 반영하는 기능도 포함한다.

1.2 핵심 차별점

기존 협업 툴(Jira, Notion, Linear 등)은 사람이 티켓을 만들고 쪼개고 배정해야 하는 "기록 도구"에 가깝다. WorkPilot AI는 다음 지점에서 다르다.

- 단순 작업 목록 관리가 아니라, AI가 요구사항을 실제 실행 단위(API/UI/연동/테스트 등)까지 분해한다.
- 배정은 규칙 기반 라운드로빈이 아니라 팀원의 스킬·과거 이력·현재 부하를 함께 고려한 추천 방식이다.
- 추적은 사람이 상태를 갱신하는 방식이 아니라, Git 커밋/PR/CI 등 개발 활동 신호를 근거로 진행률을 추정한다.
- 지연은 사후 보고가 아니라 사전 신호(커밋 정체, 마감 임박, 블로커 태그)로 감지하고, 대안 일정을 함께 제시한다.
- 회의 내용을 텍스트화·요약해 업무 맥락(결정 사항, 액션 아이템)으로 자동 반영한다.

1.3 스코프 정의 (본 문서 기준)

본 시스템은 pixel-agents(현재 코드베이스)와는 독립된 별도 프로젝트로 기획한다. pixel-agents의 레이어드 아키텍처(core/server/webview-ui 분리, Provider 추상화, AsyncAPI 계약 기반 프로토콜)는 설계 관례로 참고하고, 여기에 더해 UI는 pixel-agents의 캔버스 기반 오피스 시각화(캐릭터 FSM, 스프라이트, 타일맵, 좌석 시스템)를 실제 프론트엔드 표현 방식으로 채택한다 — 팀원을 픽셀 캐릭터로, 프로젝트를 사무실

공간으로 표현해 업무 상태를 직관적으로 보여준다(자세한 매핑은 4.2절 참조). AI 연동은 이번 단계에서 인터페이스만 설계하고, 실제 LLM 호출은 이후 단계에서 API 키를 연결해 구현한다.

2. 사용자 및 시나리오

2.1 페르소나

구분	역할	주요 니즈
PM / 팀 리드	업무를 자연어로 지시하고 전체 진행 상황을 관리	빠른 착수, 일정 가시성, 지연 조기 파악
개발자(팀원)	배정된 작업을 수행하고 진행 상황을 보고	명확한 작업 범위, 과도하지 않은 배정, 방해 최소화
이해관계자	결과 및 일정 현황을 주기적으로 확인	요약된 현황, 리스크 조기 공유

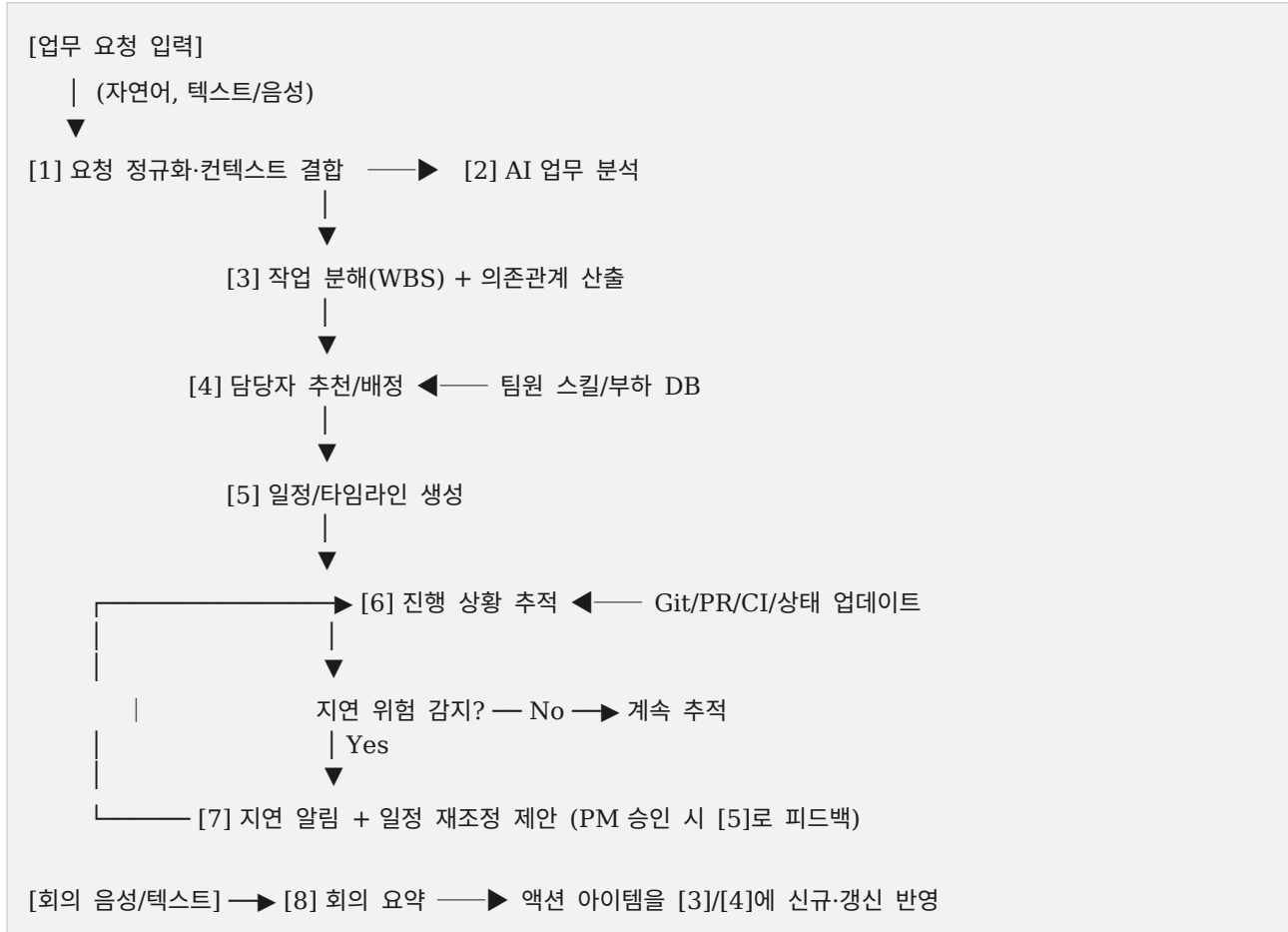
2.2 End-to-End 시나리오

예시: PM이 "회원가입 시스템을 만들고 테스트까지 진행해줘"라고 입력한 경우의 처리 흐름.

단계	AI/시스템 동작	산출물
1. 요청 접수	자연어 요청을 파싱하고 프로젝트 컨텍스트(기존 스택, 팀 구성)와 결합	정규화된 업무 요청 객체
2. 업무 분석	요구사항을 기능 단위로 분석 (회원가입 API, 화면, 로그인 연동, 테스트 등)	기능 분석 요약
3. 작업 분해 (WBS)	기능 단위를 실행 가능한 Task/Subtask로 분해, 선후행 관계 산출	WBS 트리 + 의존 그래프
4. 담당자 추천 /배정	스킬 매칭 + 부하를 고려해 팀원 추천, PM 승인 또는 자동 배정	Task별 담당자 배정
5. 일정 생성	예상 소요시간과 의존관계로 일정/간트 생성	프로젝트 타임라인
6. 진행 추적	커밋/PR/상태 업데이트를 근거로 Task 진행률 추정	실시간 진행률 대시보드
7. 지연 대응	지연 위험 감지 시 담당자 알림 + 재조정안 제시	지연 알림 + 일정 재조정 제안
8. 회의 요약	회의 음성/텍스트 입력 시 요약 및 액션 아이템 추출	회의 요약 + 신규/갱신 Task

3. 전체 프로세스 흐름

전체 파이프라인은 단방향이 아니라 진행 추적과 지연 대응 단계에서 다시 작업 분해/일정 단계로 피드백되는 순환 구조를 가진다.



4. 시스템 아키텍처

4.1 레이어 구조

pixel-agents의 레이어드 원칙(핵심 로직과 외부 연동의 분리, 프로토콜 계약 우선)을 참고하되, 독립 프로젝트에 맞게 다음과 같이 구성한다.

레이어	책임	비고
Client (Web UI)	업무 요청 입력, 픽셀 오피스 시각화(팀원=캐릭터, Task 배정=좌석/타이핑 상태), WBS/간트는 보조 패널, 알림 확인	Canvas 2D 렌더링 + React 오버레이 UI
API Gateway	인증, 요청 라우팅, Rate limit	REST +

레이어	책임	비고
		WebSocket(실시간 진행률)
Orchestration / AI Engine	요청 분석, 작업 분해, 배정 추천, 지연 분석, 회의 요약 오케스트레이션	LLM Provider 인터페이스를 통해 호출 (현 단계는 Mock)
Domain Services	Task/Project/Schedule/Member/Notification 도메인 로직	핵심 비즈니스 규칙, LLM에 비의존
Integration Adapters	Git/GitHub·GitLab, Slack, 캘린더, 음성 전사 (STT) 등 외부 연동	Provider 패턴 — 신규 연동은 어댑터 추가만으로 확장
Data Store	프로젝트/작업/일정/팀원/회의록 영속화	RDB(PostgreSQL) + 벡터 저장(회의록 /요구사항 검색)

4.2 UI 컨셉: 픽셀 오피스 시각화

팀원과 진행 상황을 표/리스트가 아니라 pixel-agents와 같은 픽셀 아트 오피스로 표현한다. PM은 팀을 관장하는 Lead 캐릭터로, 각 팀원은 고유 팔레트를 가진 Teammate 캐릭터로 표시되며, 좌석(책상)은 담당 프로젝트/역할에 따라 배치한다. 캐릭터의 상태 전이는 pixel-agents의 캐릭터 FSM(idle/active/typing 등)과 말풍선 시스템을 그대로 차용해 Task 상태에 매핑한다.

PM 업무 자동화 개념	픽셀 오피스 시각 요소	동작 방식
PM	Lead 캐릭터	팀 전체를 관장하는 리드로 표시, 팀 리더 배치
팀원(Member)	Teammate 캐릭터	고유 팔레트 배정, 담당 프로젝트의 좌석(책상) 보유
Task 배정 + 진행중	Active 상태(자리에 앉아 Type 애니메이션)	커밋/PR 등 진행 신호가 있으면 타이핑, 신호 없으면 idle로 전환
Subtask	부모 곁에 스폰되는 Sub-agent 캐릭터	병렬로 진행되는 하위 작업을 별도 캐릭터로 시각화
AI 추천 배정/재조정 승인 대기	"..." Permission 말풍선(지속)	PM이 클릭해 승인/거절할 때까지 유지
Task 완료	체크마크 Waiting 말풍선(2초 페이드)	완료 즉시 표시 후 자동 소멸

PM 업무 자동화 개념	픽셀 오피스 시각 요소	동작 방식
자연 위험 감지	경고색 Alert 말풍선(지속) + 사운드 알림	담당자/PM이 확인 클릭하기 전까지 유지
신규 프로젝트 착수/팀원 합류	Spawn 이펙트(매트릭스 레인)	캐릭터가 사무실에 새로 등장
프로젝트 종료/팀원 이탈	Despawn 이펙트	캐릭터가 사무실에서 퇴장
Task 진행률	캐릭터 위 진행률 게이지	컨텍스트 게이지와 동일한 오버레이 개념을 진행률(%)로 재활용
회의 진행/요약 완료	회의실 가구 구역에 캐릭터 집합 + 요약 알림 말풍선	액션 아이템이 신규 Task로 배정되면 담당 캐릭터에게 전파

적용 범위와 보조 뷰

픽셀 오피스는 팀 규모가 작을 때(대략 10~15명 이하) 가장 효과적이다. 팀이 커지거나 세부 일정을 정밀하게 봐야 할 때를 대비해 WBS 트리, 간트 차트, 리스트형 대시보드를 보조 뷰(패널 전환)로 함께 제공한다 — 두 뷰는 같은 AgentStateStore 개념의 단일 상태를 공유해 서로 어긋나지 않는다.

4.3 핵심 설계 원칙

- Domain Services는 어떤 LLM을 쓰는지 알지 못한다 — AI Engine이 결과를 표준 스키마(Task, Assignment, Schedule 등)로 변환해 넘긴다.
- 모든 외부 연동(Git, Slack, STT, 캘린더)은 Adapter 인터페이스 뒤에 위치해 신규 도구 추가 시 오케스트레이션/도메인 로직 변경이 없다.
- AI 판단(분해, 배정 추천, 자연 원인 분석)은 항상 근거(예: 참조한 커밋, 유사 과거 Task)와 함께 반환되어 PM이 승인/수정할 수 있어야 한다 — 전 구간 자동 확정 금지.
- 실시간 진행률/알림은 WebSocket으로 push하고, 이력은 REST로 조회하는 이중 채널 구조.

5. 핵심 기능 상세 설계

5.1 업무 요청 접수 및 분석

PM의 자연어 요청을 프로젝트 컨텍스트(기존 아키텍처, 기술 스택, 팀 구성, 과거 유사 작업 이력)와 함께 AI Engine에 전달한다. 1차 산출물은 "기능 분석 요약"으로, 범위에 포함/제외되는 항목을 PM이 즉시 확인·수정할 수 있게 한다.

입력 예시

"회원가입 시스템을 만들고 테스트까지 진행해줘"

분석 결과 예시(요약)

- 포함: 회원가입 API, 회원가입 UI, 이메일 인증, 로그인 연동, 단위/통합 테스트
- 불확실(확인 필요): 소셜 로그인 포함 여부, 비밀번호 정책, 이메일 발송 인프라 기존 여부
- 제외(명시적 언급 없음): 회원 탈퇴, 관리자 페이지

5.2 작업 분해 (WBS 생성)

기능 분석 결과를 실행 가능한 Task 단위로 분해한다. 각 Task는 예상 소요시간, 선행 Task, 필요 스킬 태그를 함께 산출한다.

Task	선행 작업	예상 소요	필요 스킬
회원가입 API 설계/구현	-	2일	Backend, DB
회원가입 UI 구현	API 설계(스펙 확정 시점)	1.5일	Frontend
이메일 인증 연동	회원가입 API	1일	Backend, 외부연동
로그인 연동	회원가입 API	1일	Backend, Frontend
단위 테스트 작성	각 기능 구현 완료	1일	Test
통합 테스트 및 QA	전체 기능 완료	1.5일	QA

5.3 담당자 추천 및 배정

배정 추천은 다음 세 요소의 가중합으로 산출한다: 스킬 적합도, 현재 배정된 작업량(부하), 과거 유사 Task 수행 이력(속도/품질 신호). PM은 추천안을 그대로 승인하거나 수동으로 재배정할 수 있다.

```
score(member, task) =
  w1 * skillMatch(member.skills, task.requiredSkills)
+ w2 * (1 - normalizedCurrentLoad(member))
+ w3 * pastPerformance(member, task.category)
```

기본 가중치(튜닝 가능): w1=0.5, w2=0.3, w3=0.2

5.4 일정 생성

Task 간 의존관계와 예상 소요시간, 팀원별 가용 시간(캘린더 연동)을 바탕으로 Critical Path를 계산해 프로젝트 타임라인(간트)을 생성한다. 팀원 휴가/다른 프로젝트 배정도 가용 시간 계산에 반영한다.

5.5 진행 상황 추적

Task 상태는 사람이 수동으로 바꾸는 것에만 의존하지 않는다. 다음 신호를 결합해 진행률을 추정하고, 신호가 일정 기간 없으면 정체로 간주한다.

신호 소스	반영 방식
Git 커밋/브랜치 활동	Task와 연결된 브랜치·커밋 메시지 패턴(Task ID 태깅)으로 활동 감지
PR 상태	PR Open/Review/Merge 상태를 Task 진행 단계에 매핑
CI 결과	테스트 Task의 완료 신호로 CI 통과 여부 활용
수동 상태 업데이트	개발자가 직접 남기는 상태/코멘트, 최우선 신호로 취급

5.6 지연 감지 및 대응

지연 위험은 사후 보고가 아니라 다음 규칙으로 사전 감지한다.

- 예상 소요시간 대비 경과 시간이 임계치(예: 80%)를 초과했는데 진행 신호가 없는 경우
- 연속 N일간 관련 커밋/PR 활동이 없는 경우
- 선행 Task 지연으로 후행 Task 착수가 밀리는 경우(의존관계 전파)

위험이 감지되면 해당 담당자에게 알림을 보내고, 동시에 AI가 대안 일정(예: 범위 축소, 지원 인력 투입, 후속 일정 순연)을 제시한다. PM 승인 시 일정이 갱신되고 영향받는 후행 Task/담당자에게 자동 공지된다.

5.7 회의 음성/텍스트 요약

회의 음성 파일 또는 텍스트를 입력하면 STT(음성인 경우) 후 AI가 요약한다. 요약 결과는 결정 사항, 액션 아이템, 리스크/이슈로 구조화되며, 액션 아이템은 신규 Task 생성 또는 기존 Task 갱신 제안으로 이어진다.

- 결정 사항: 회의에서 확정된 방향/정책
- 액션 아이템: 담당자·기한이 유추되는 후속 작업 (승인 시 Task로 전환)
- 리스크/이슈: 후속 확인이 필요한 미결 사항

6. 데이터 모델 (개념 설계)

엔티티	주요 속성	관계
Project	id, name, stack, startDate, targetDate	1:N Task
Task	id, title, description, status, requiredSkills, estimateHours, dependsOn[]	N:1 Project, N:1 Member(assignee), 1:N ProgressSignal
Member	id, name, skills[], currentLoad, pastPerformance	1:N Task(assigned)
Schedule	id, projectId, taskId, plannedStart, plannedEnd, actualStart, actualEnd	1:1 Task
ProgressSignal	id, taskId, source(git/pr/ci/manual), payload, timestamp	N:1 Task
DelayAlert	id, taskId, detectedAt, reason, proposedReschedule, status	N:1 Task
MeetingNote	id, projectId, date, rawText/audioRef, summary, decisions[], actionItems[]	1:N Task(생성/갱신 연결)
Notification	id, targetMemberId, type, payload, readAt	N:1 Member

7. AI 연동 인터페이스 설계 (Mock 우선)

이번 단계에서는 실제 LLM API 키 연동 없이, 이후 Claude API 등으로 바로 교체 가능한 Provider 인터페이스만 설계한다. Domain Services는 아래 인터페이스에만 의존하며, Mock 구현과 실제 구현을 설정으로 교체한다(pixel-agents의 HookProvider 추상화와 동일한 사상).

7.1 AIProvider 인터페이스

```
interface AIProvider {
  analyzeRequest(input: RawRequest, context: ProjectContext): Promise<RequestAnalysis>;
  decomposeIntoTasks(analysis: RequestAnalysis): Promise<TaskDraft[]>;
  recommendAssignee(task: TaskDraft, members: Member[]): Promise<AssigneeRecommendation[]>;
  analyzeDelayRisk(task: Task, signals: ProgressSignal[]): Promise<DelayAssessment | null>;
  summarizeMeeting(input: MeetingInput): Promise<MeetingSummary>;
}
```

// 구현체 예

```
class MockAIProvider implements AIProvider { /* 규칙 기반 더미 응답 */ }
```

```
class ClaudeAIProvider implements AIProvider { /* Anthropic API 호출 (후회 단계) */ }
```

7.2 프롬프트 템플릿 설계 예시 — 작업 분해

SYSTEM:

당신은 소프트웨어 개발 PM 보조 AI입니다. 아래 기능 분석 결과를 실행 가능한 Task 목록으로 분해하세요. 각 Task는 title, description, requiredSkills, estimateHours, dependsOn(선행 Task 제목 배열)을 포함한 JSON 배열로만 응답하세요.

CONTEXT:

- 프로젝트 스택: {{stack}}
- 팀 구성: {{teamSkillSummary}}

INPUT:

{{requestAnalysis}}

7.3 Mock 모드 동작 방식

MockAIProvider는 실제 LLM 호출 없이 규칙 기반(키워드 매칭 + 사전 정의된 템플릿)으로 동일한 응답 스키마를 반환한다. 이를 통해 API 키 없이도 전체 파이프라인(요청→분해→배정→일정→추적)을 end-to-end로 검증할 수 있으며, 이후 ClaudeAIProvider로 설정만 교체하면 실제 AI 판단으로 전환된다.

8. 기술 스택 제안

영역	제안 스택	비고
Frontend	React + TypeScript + Canvas 2D 렌더링 엔진 (pixel-agents webview-ui의 office 렌더러 개념을 포크: 스프라이트 캐시, 캐릭터 FSM, 타일맵, 좌석 시스템). WBS/간트/리스트는 보조 패널.	실시간 캐릭터 상태· 말풍선은 WebSocket 이벤트로 구동
Backend	Node.js + TypeScript (Fastify)	pixel-agents와 스택 친화성, 팀 러닝커브 최소화
AI 연동	AIProvider 인터페이스 + Anthropic Claude API(추후)	이번 단계는 MockAIProvider만 구현
DB	PostgreSQL	관계형 데이터(Task/Schedule/ Member) 중심
회의 STT	외부 STT API 연동 어댑터	음성 입력 시에만 사용, 텍스트 입력은 STT 생략
외부 연동	GitHub/GitLab API, Slack API, 캘린더 API	Adapter 패턴으로 신규 연동 확장

9. MVP 로드맵

Phase	목표	주요 범위
Phase 1	핵심 파이프라인 + 기본 픽셀 오피스 UI (Mock AI)	요청 입력 → 분해(WBS) → 수동 승인 배정 → 정적 일정. 팀원을 픽셀 캐릭터로 표시하고 좌석 (책상)에 배정(정적 상태). AI는 MockAIProvider.
Phase 2	진행 추적 + 지연 감지 + 캐릭터 상태 연동	Git/PR 연동, 캐릭터 idle/active 전환, 승인 대기 /완료/지연 말풍선, 진행률 게이지, 규칙 기반 지연 감지.
Phase 3	실제 LLM 연동	ClaudeAIProvider 구현, 프롬프트 튜닝, 배정 추천 고도화(이력 기반).
Phase 4	회의 요약 + 자동 재조정 + 연출 고도화	STT 연동, 회의 요약→Task 반영, 지연 시 일정 재조정 자동 제안, 스폰/디스폰 이펙트·회의실 연출 추가.

10. 리스크 및 고려사항

- AI 분해/배정 결과의 신뢰도: 초기에는 PM 승인 단계를 필수로 두어 오판 리스크를 완화해야 한다.
- 진행률 추정의 정확도: 커밋/PR만으로는 실제 진척과 괴리가 있을 수 있어 수동 업데이트를 최우선 신호로 병행해야 한다.
- 외부 연동 의존성: Git/Slack/캘린더/STT 등 다수 외부 API에 의존 — 각 연동은 장애 시에도 핵심 파이프라인이 동작하도록 Adapter 단에서 격리.
- 개인정보/보안: 회의 내용, 커밋 로그 등 민감 정보를 다루므로 저장·전송 구간 암호화 및 접근 권한 관리가 필요.
- 과도한 자동화에 대한 팀 반발: 배정/일정 변경은 항상 근거를 제시하고 사람이 최종 승인하는 구조를 유지해 신뢰를 확보해야 한다.
- 픽셀 오피스 UI의 가독성 한계: 팀 규모가 커지면(수십 명) 캐릭터 시각화만으로는 한눈에 파악하기 어려워 목록/테이블형 보조 뷰를 함께 제공해야 한다.

11. 성공 지표 (KPI 안)

지표	정의
업무 착수 소요 시간	요청 입력 시점부터 WBS/배정/일정 확정까지 걸리는 시간
지연 조기 감지율	실제 지연 발생 건 중 사전 알림이 선행된 비율
배정 재조정 비율	AI 추천 배정이 PM에 의해 수동 변경되는 비율(낮을수록 추천 신뢰도 높음)
회의 액션 아이템 반영률	회의 요약에서 추출된 액션 아이템이 실제 Task로 전환·완료되는 비율